Compte rendu

Application Vue.js de gestion des contacts

Étudiant	***Aboubacar BAH***
Classe / Groupe	L2DSI1
Enseignant	TAHAR HAOUET
Année universitaire	2026-2027
Dépôt GitHub	https://github.com/bahaboubacar8958-tech/Gestion_de_Contact

Sommaire

- 1. Introduction et objectifs
- 2. Environnement et outils utilisés
- 3. Architecture du projet
- 4. Modèle de données et gestion de l'état
- 5. Page ListeContacts
- 6. Page DetailContact
- 7. Utilisation des fonctionnalités du navigateur
- 8. Validation du formulaire
- 9. Tests effectués
- 10. Captures d'écran
- 11. Difficultés rencontrées et solutions
- 12. Conclusion

1. Introduction et objectifs

Ce travail pratique consiste à réaliser une application web de gestion de contacts avec Vue.js. L'application permet de consulter une liste de contacts (avec photo, nom, prénom et numéro de téléphone), d'ouvrir la fiche d'un contact pour la modifier, d'ajouter un nouveau contact et d'utiliser les fonctionnalités natives de l'appareil : appel téléphonique, envoi d'e-mail, choix d'une photo depuis la galerie et prise de photo avec la caméra.

Les objectifs pédagogiques sont les suivants :

- maîtriser les composants Vue.js et le routage avec Vue Router ;
- partager un état réactif entre plusieurs pages ;
- manipuler les formulaires, leur validation et les prévisualisations d'images ;
- exploiter les API et liens natifs du navigateur (tel:, mailto:, input file, capture).

2. Environnement et outils utilisés

Outil	Rôle
Vue.js 3 (Composition API)	Framework JavaScript pour la construction de l'interface réactive
Vue Router 4	Navigation entre les pages ListeContacts et DetailContact
Vite	Création et lancement du projet (serveur de développement, build)
Node.js / npm	Environnement d'exécution et gestionnaire de paquets
Visual Studio Code	Éditeur de code
Git / GitHub	Gestion de versions et dépôt du code source
Navigateur (Chrome, appareil mobile)	Exécution et tests de l'application

3. Architecture du projet

Le projet est organisé en composants réutilisables, avec une séparation claire entre les pages, les composants d'affichage et la gestion des données.

```
src/
|-- main.js                 (point d'entrée, montage de l'app et du routeur)
|-- App.vue                 (en-tête + <router-view>)
|-- router/index.js         (routes : /, /contact/:id, /ajouter)
|-- store/contacts.js       (état partagé des contacts et actions)
|-- views/
|    |-- ListeContacts.vue  (page liste)
|    '-- DetailContact.vue  (page fiche / modification)
'-- components/
     |-- ContactItem.vue    (ligne de contact : photo, nom, prénom, tél.)
     |-- ContactAvatar.vue  (photo ou initiales de repli)
     '-- PhotoPicker.vue    (Galerie / Caméra / prévisualisation)
```

Routage. Trois routes sont définies : la liste (`/`), la fiche d'un contact existant (`/contact/:id`) et la création d'un nouveau contact (`/ajouter`). Le bouton « Modifier » transmet l'identifiant du contact dans l'URL, et le bouton « Retour » ramène à la liste.

4. Modèle de données et gestion de l'état

4.1 Structure d'un contact

Champ	Type	Description
id	Nombre / chaîne	Identifiant unique du contact
nom	Chaîne	Nom de famille (obligatoire)
prenom	Chaîne	Prénom (obligatoire)
telephone	Chaîne	Numéro de téléphone (obligatoire)
email	Chaîne	Adresse e-mail (obligatoire, format valide)
photo	Chaîne (URL ou Data URL)	Image du contact, affichée en miniature dans la liste et en grand dans la fiche

La liste initiale contient **au moins cinq contacts distincts**, chacun avec sa propre photo, afin de vérifier l'affichage des miniatures directement dans la liste.

4.2 Partage de l'état entre les pages

Les contacts sont stockés dans un état réactif unique (`reactive` / `ref`) défini dans un module partagé et importé par les deux pages. Ce module expose la liste des contacts ainsi que les actions `getContact(id)`, `addContact()` et `updateContact()`. Grâce à la réactivité de Vue, toute modification enregistrée depuis DetailContact est immédiatement visible dans ListeContacts, **sans rechargement de la page**.

En bonus, l'état est sauvegardé dans le `localStorage` du navigateur : les contacts et leurs photos sont donc conservés après un rechargement.

5. Page ListeContacts

Cette page affiche tous les contacts sous forme de liste. Chaque ligne, rendue par le composant `ContactItem` via une boucle `v-for`, contient :

- la **photo** du contact en miniature (visible directement dans la liste, et non uniquement dans la fiche) ;
- le **nom** et le **prénom** ;
- le **numéro de téléphone** ;
- un bouton **« Modifier »** qui ouvre la fiche du contact sélectionné.

Un bouton **« + Ajouter »** placé en haut de la page ouvre un formulaire vide pour créer un nouveau contact. Après l'enregistrement, le contact apparaît immédiatement dans la liste.

Si un contact n'a pas de photo, un cercle coloré avec l'initiale est affiché à la place (composant `ContactAvatar`).

6. Page DetailContact

La fiche permet de consulter et de modifier un contact. Les champs sont liés au formulaire par `v-model` sur une **copie locale** du contact : ainsi, un clic sur « Retour » sans enregistrement n'altère pas les données de la liste.

Bouton	Fonctionnement
Appeler	Lien `tel:` + numéro : ouvre l'application téléphonique de l'appareil
E-mail	Lien `mailto:` + adresse : ouvre le logiciel de messagerie avec le destinataire pré-rempli
Galerie	Champ `input type="file" accept="image/*"` pour choisir une image dans l'appareil
Caméra	Champ `input type="file" accept="image/*" capture` pour prendre une photo sur mobile, avec solution de repli
Enregistrer	Valide le formulaire puis met à jour le contact dans l'état partagé
Retour	Revient à la liste via Vue Router, sans enregistrer les modifications en cours

7. Utilisation des fonctionnalités du navigateur

Prévisualisation de la photo. Lorsqu'un fichier est choisi, il est lu avec `FileReader.readAsDataURL()` (ou `URL.createObjectURL()`) et le résultat est affiché immédiatement dans l'aperçu, avant même l'enregistrement. Le type du fichier est vérifié (image uniquement).

Prise de photo. Sur un téléphone compatible, l'attribut `capture` ouvre directement l'appareil photo. Sur un ordinateur ou un navigateur ne prenant pas en charge la capture, le champ se comporte comme un sélecteur de fichier classique ; un message informe alors l'utilisateur que la caméra n'est pas disponible et l'invite à utiliser la galerie. Ce comportement constitue la solution de repli demandée.

Appel et e-mail. Ces actions reposent uniquement sur les schémas d'URL `tel:` et `mailto:` : aucune bibliothèque externe n'est nécessaire.

8. Validation du formulaire

Avant l'enregistrement, les contrôles suivants sont effectués :

- nom, prénom, téléphone et e-mail ne doivent pas être vides (espaces seuls refusés) ;
- l'adresse e-mail doit respecter un format valide (expression régulière du type `texte@domaine.extension`) ;
- en cas d'erreur, un message est affiché sous le champ concerné et l'enregistrement est bloqué.

9. Tests effectués

N°	Test	Résultat attendu	Résultat
1	Affichage de la liste initiale	Les 5 contacts affichent photo, nom, prénom et téléphone	Conforme
2	Modification d'une fiche puis enregistrement	Les changements apparaissent dans la liste sans rechargement	Conforme
3	Ajout d'un nouveau contact	Le contact est ajouté à la liste	Conforme
4	Bouton Appeler	L'application téléphonique s'ouvre avec le numéro	Conforme
5	Bouton E-mail	Le client de messagerie s'ouvre avec le destinataire	Conforme
6	Photo depuis la galerie	Aperçu immédiat, puis photo mise à jour dans la liste	Conforme
7	Photo avec la caméra (téléphone)	L'appareil photo s'ouvre, la photo est affichée	Conforme
8	Caméra indisponible (ordinateur)	Message d'information et repli vers la galerie	Conforme
9	Champs vides	Message d'erreur, enregistrement bloqué	Conforme
10	E-mail invalide	Message d'erreur, enregistrement bloqué	Conforme
11	Retour sans enregistrer	Les modifications sont ignorées	Conforme
12	Rechargement de la page (bonus)	Les données sont conservées grâce au localStorage	Conforme

10. Captures d'écran

Les captures d'écran de l'application réalisée sont regroupées dans le fichier PDF nommé **Capture_d'écran.pdf**, joint à ce compte rendu dans le dépôt GitHub. Elles montrent : la liste des contacts avec les photos, la fiche détaillée, la modification d'un contact, la sélection d'une photo depuis la galerie, la prise de photo avec la caméra et la liste mise à jour après l'enregistrement.

11. Difficultés rencontrées et solutions

Difficulté	Solution apportée
Partage des données entre les deux pages	Création d'un module d'état réactif unique importé par les deux vues
Modifications conservées malgré un retour sans enregistrement	Utilisation d'une copie locale du contact dans le formulaire
Prévisualisation de l'image avant enregistrement	Lecture du fichier avec FileReader et affichage de la Data URL
Caméra non disponible sur ordinateur	Attribut capture avec repli vers la galerie et message d'information
Persistance des photos après rechargement	Sauvegarde des contacts (photos en Data URL) dans le localStorage

12. Conclusion

Ce TP a permis de réaliser une application complète de gestion de contacts avec Vue.js et Vue Router. Toutes les fonctionnalités demandées ont été mises en place : liste avec photos, fiche modifiable, validation des champs, appel, e-mail, choix de photo depuis la galerie et prise de photo avec solution de repli. Il a également permis de mieux comprendre la réactivité de Vue, la découpe en composants et l'utilisation des fonctionnalités natives du navigateur.

Pistes d'amélioration : suppression de contact, recherche et tri dans la liste, utilisation de Pinia, stockage des données sur un serveur ou une base de données.